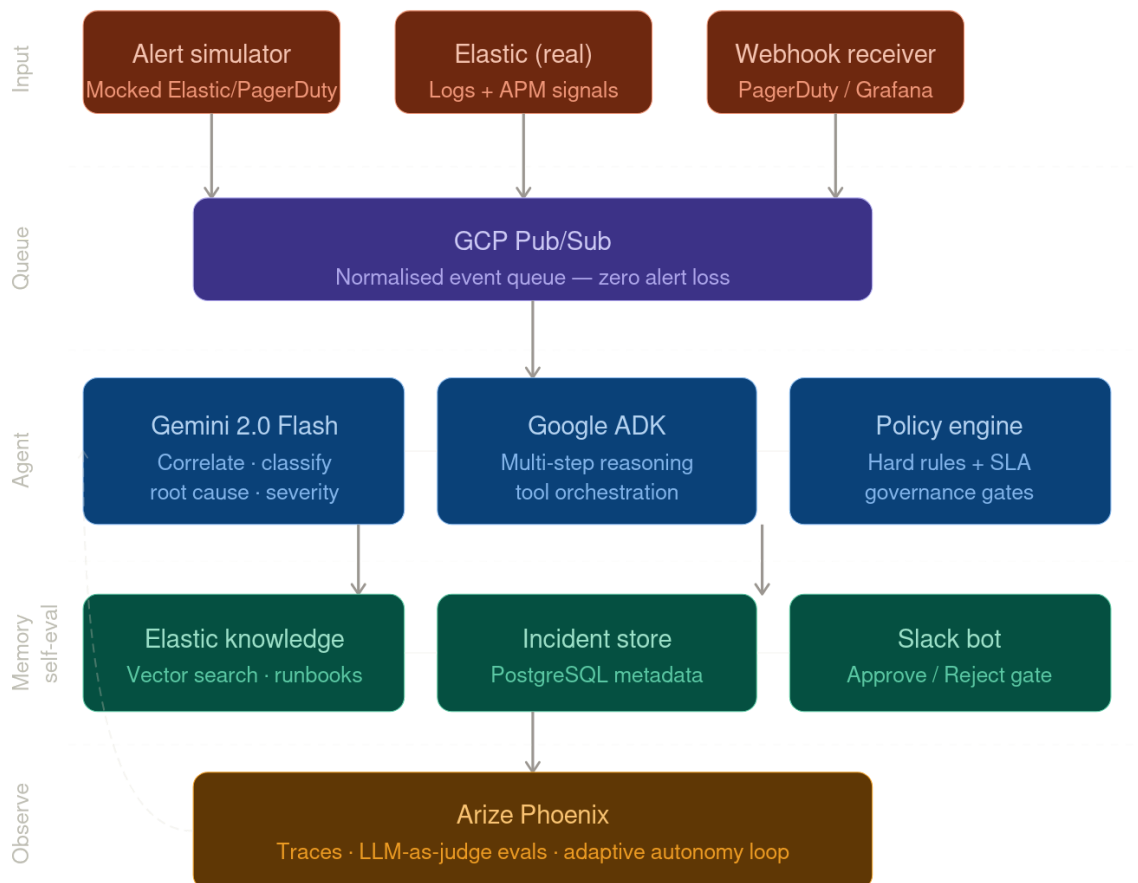


OpsSentinel: Master Architecture and Minimum Viable Product Specification



Executive Overview and Industry Context

The enterprise IT operations landscape in the year 2026 is characterized by deeply fragmented automation architectures. While organizations rely heavily on sophisticated monitoring tools, incident response times remain suboptimal due to the lack of a cohesive, intelligent orchestration layer.¹ The following document serves as the foundational architectural specification and Minimum Viable Product (MVP) blueprint for OpsSentinel—a self-aware, agentic AI platform designed to correlate cross-system alerts, orchestrate human-in-the-loop remediation, and continuously evaluate its own operational reliability.¹ This specification synthesizes advanced system design principles, Google Cloud deployment best practices, and

the integration of the Model Context Protocol (MCP) via Google's Agent Development Kit (ADK).²

Modern enterprises possess a multitude of distinct automation islands, ranging from PagerDuty for notifications and ServiceNow for ticketing, to Elastic and Datadog for anomaly detection.¹ Despite these systems, structural inefficiencies persist. Recent data indicates that 63% of generated alerts remain unaddressed daily, and 40% of engineering time is consumed by incident firefighting rather than feature development.¹ The fundamental problem is not a lack of diagnostic data, but the absence of centralized, cross-platform reasoning. The architecture of modern incident response suffers from fragmented automation, where discrete tools detect and route anomalies but fail to communicate intelligently, leaving human operators to manually correlate signals across multiple separate dashboards.¹

Furthermore, existing AIOps platforms react to isolated, immediate events without contextual agentic memory of prior incidents, successful resolution paths, or specific environmental configurations.¹ They also suffer from deficient self-evaluation capabilities. Current generative AI copilots issue recommendations with static confidence levels, lacking the capacity to review their own historical track records, detect model drift, or calibrate their autonomy based on empirical accuracy.¹ Finally, a single microservice failure often triggers hundreds of redundant alerts across monitoring tools, creating cascading alert storms. Without a system capable of semantic deduplication and correlation, engineering teams are inundated with phantom issues stemming from a single root cause.¹

From a system design perspective, handling these alert storms requires a precise understanding of the dichotomy between performance and scalability.² A system exhibits a performance bottleneck if it is slow for a single user, whereas a scalability bottleneck occurs when a system is fast for a single user but degrades under heavy load.² In cascading failure scenarios, existing ticketing systems face scalability bottlenecks, processing hundreds of redundant alerts sequentially rather than dynamically scaling to correlate and synthesize them into a single, actionable vector.¹ OpsSentinel addresses this by introducing a highly scalable event orchestration layer capable of resolving these throughput challenges while maintaining acceptable latency.²

Target Users and Operational Personas

The platform is engineered to serve specialized technical cohorts responsible for maintaining high-availability distributed systems.¹ The psychological and operational toll on these professionals is immense, largely driven by context switching at irregular hours and the cognitive load of parsing unstructured machine data during high-stakes outages.¹ Table 1 delineates the primary user personas, their operational pain points, and the platform's proposed resolutions.

Operational Persona	Core Pain Point	Systemic Goal and Resolution

Site Reliability Engineer (SRE)	Severe alert fatigue, manual cross-tool correlation, and extreme context switching at irregular hours.	Faster incident triage via automated alert correlation, unified decision briefs, and single-click execution of historical fixes. ¹
DevOps Pipeline Engineer	Highly manual root-cause analysis for CI/CD deployment regressions and pipeline failures.	Automated log analysis connecting code commits to runtime regressions, enabling targeted and immediate rollbacks. ¹
Engineering Manager / Director	Complete lack of visibility into automation reliability and operational risk across the stack.	Real-time observability dashboards demonstrating AI decision accuracy, calibration, and overall operational health. ¹
Platform Infrastructure Lead	Overwhelming cascading alert storms during Kubernetes cluster or network partitioning events.	Consolidation of hundreds of cascading alerts into a single root-cause incident via semantic correlation algorithms. ¹

Core Hypotheses and Strategic Validation

The development of OpsSentinel is predicated on the validation of specific, high-value technical hypotheses. Building complex features without validating these assumptions leads to misaligned product trajectories and uncontrolled scope creep. The primary assumption being validated is the Runtime Adaptive Autonomy Hypothesis.¹ This hypothesis posits that if an AI agent continuously queries its own performance traces to calibrate its confidence score, enterprise risk management teams will trust the system to execute autonomous remediations at a significantly higher frequency.¹ Without this self-evaluating safety mechanism, enterprises will continue to restrict AI to read-only advisory roles.

The second foundational assumption is the Contextual Memory Hypothesis. This assumes that if incident resolution steps and post-mortem analyses are embedded in a semantic vector database and dynamically retrieved during similar future anomalies, Mean Time to Resolution (MTTR) will decrease exponentially.¹ The system relies on the assumption that historical context is a more reliable predictor of successful remediation than zero-shot generative reasoning. Finally, the MCP Standardization Hypothesis asserts that utilizing the Model Context Protocol (MCP) to interact with localized tools like Elasticsearch and Arize Phoenix will significantly reduce the latency, maintenance overhead, and brittleness associated with legacy API wrapper

integrations.⁴

Success Metrics and Performance Thresholds

Quantifiable success parameters dictate the viability of the Minimum Viable Product. These metrics are strictly categorized into business outcomes, operational efficiency, and underlying system design thresholds.¹ Vague metrics such as general user satisfaction are discarded in favor of empirical telemetry. Table 2 outlines the rigorous success criteria for the MVP.

Metric Classification	Target Indicator	Success Threshold
Operational Velocity	Mean Time to Detect (MTTD)	The analytical engine must complete correlated analysis within 30 seconds of initial signal detection. ¹
Operational Velocity	Mean Time to Resolution (MTTR)	Reduction of MTTR to under 10 minutes for known incident patterns, down from the industry average of 40-90 minutes. ¹
AI Efficacy	Triage and Routing Accuracy	Over 90% accuracy in correctly categorizing the incident, determining severity, and assigning the proper remediation team. ¹
AI Efficacy	Alert Correlation Precision	Achieve an 85% reduction in duplicate P1/P2 tickets during cascading failure events by grouping related signals. ¹
Human Trust	Autonomous Approval Rate	Exceed an 80% first-pass approval rate from human engineers for AI-generated remediation strategies without modification. ¹
System Reliability	Arize Confidence Calibration	The variance between the agent's stated confidence

		and its empirical accuracy must remain below a 5% margin of error. ¹
--	--	---

From a systemic perspective, the architecture must also achieve maximal throughput for alert ingestion while maintaining sub-second latency for message queuing, ensuring real-time responsiveness during high-load alert storms.² The MVP architecture targets 99.99% availability, translating to a maximum acceptable downtime of 52 minutes and 35.7 seconds per year, or 8.6 seconds per day.²

MVP Scope and Strict Constraints

To ensure a high probability of success within constrained development timelines, features are strictly bifurcated into essential parameters and explicitly deferred capabilities.¹ This enforces lean MVP principles, preventing scope creep and focusing engineering resources on validating the core hypotheses. Table 3 delineates the MVP boundaries.

Category	Component	Description and Justification
Must Have (V1)	Signal Orchestrator	Ingestion of webhooks from existing monitoring platforms via a scalable event bus to prevent alert loss. ¹
Must Have (V1)	Gemini Reasoning Engine	Multi-step contextual analysis using Gemini 2.0 Flash to synthesize logs, deployment data, and service dependencies. ¹
Must Have (V1)	Elastic Vector Retrieval	Integration with Elasticsearch via MCP to execute semantic searches against historical incident runbooks. ¹
Must Have (V1)	Arize Phoenix Tracing	Implementation of OpenInference instrumentation to record every reasoning span, tool

		call, and token usage parameter. ¹
Must Have (V1)	Slack Human-in-the-Loop	Delivery of plain-text decision briefs to Slack, complete with calculated risk levels and binary execution capabilities. ¹
Deferred (Post-MVP)	Multi-Agent Collaboration	Deferring the implementation of discrete, specialized sub-agents in favor of a single generalized orchestrator. ¹
Deferred (Post-MVP)	Digital Twin Simulation	Executing remediations in a live virtual replica of production prior to deployment is deferred to mature iterations. ¹
Deferred (Post-MVP)	Predictive Prevention	Forecasting anomalies before they breach thresholds remains outside the MVP boundaries. ¹

The MVP is bound by several non-negotiable constraints. It will operate exclusively within a single cloud provider environment (Google Cloud Platform) to minimize networking complexity. Incident resolution capabilities will initially be constrained to Kubernetes-based infrastructure failures, providing a focused training distribution for the models.¹ Furthermore, inference costs must be strictly capped per day, utilizing Google Cloud Scheduler to enforce hard limits on API invocations during runaway alert storms. The system will solely support English-language log parsing and runbook generation for the V1 release.

User Journey and Workflow Dynamics

The operational workflow defines the exact sequence of events triggered by a production anomaly. A meticulously defined workflow ensures deterministic code execution and prevents chaotic microservice interactions. The user journey begins at 2:47:00 AM, when a database connection pool is exhausted following a flawed midnight deployment. In a legacy setup, Elastic detects the error spike, PagerDuty fires a concurrent webhook, and ServiceNow generates a blank ticket, leaving the engineer to manually piece the narrative together.¹

Under the OpsSentinel architecture, the workflow is fundamentally transformed. The system

ingests the asynchronous signals from all three platforms simultaneously. Instead of creating parallel execution threads, the signal orchestrator correlates the temporal and spatial data, bundling them into a singular incident context.¹ The ADK-orchestrated Gemini model ingests this correlated alert and issues a tool call via the Elastic MCP server to retrieve the last thirty minutes of application logs alongside the closest historical incident match via vector search.¹ Crucially, prior to finalizing a recommendation, the agent utilizes the Arize Phoenix MCP server to query its recent historical accuracy for "Database Connection Pool" incidents.¹ Determining a 91% historical accuracy for this category, the agent generates an execution brief proposing a connection pool restart and dynamic limit increase.¹ At 2:48:00 AM, the on-call engineer receives a structured Slack notification detailing the root cause, historical precedent, and the proposed fix.¹ The engineer clicks the approval action, prompting the system to execute the webhook to restart the pool, update the ServiceNow ticket, and permanently log the successful outcome into the Elastic knowledge base and the Arize trace history.¹ This reduces a typical forty-minute manual diagnostic process into a streamlined two-minute verification task.

Master Architecture and System Design

The high-level architecture is designed to manage high-throughput event ingestion while ensuring reliable, low-latency execution of complex AI workflows. The system adheres to advanced principles of modularity, asynchronous processing, and high availability, specifically structured across five distinct processing layers.

The Input Layer serves as the primary ingress point for all telemetry. This includes an alert simulator used for mock Elastic and PagerDuty signals during testing, alongside live webhook receivers configured to ingest real APM signals and PagerDuty routing alerts. To manage the immense volume of data generated during a cascading failure, this layer must prioritize throughput over localized latency.²

The Queue Layer acts as the critical shock absorber for the system. Incoming data from the input layer is immediately routed into Google Cloud Pub/Sub, a highly durable messaging queue.¹ During an alert storm, thousands of monitoring tools may fire simultaneously. Processing these synchronously would rapidly deplete memory and compute resources, leading to dropped alerts. Pub/Sub provides a normalized event queue that guarantees zero alert loss, absorbing the inbound traffic spike and applying back-pressure to the processing nodes.¹ This ensures that the downstream reasoning engine consumes events at a sustainable, controlled rate, adhering to the principle of asynchronous decoupling in distributed systems.² The Agent Layer houses the core cognitive processing components. Events pulled from the Pub/Sub queue are fed into the Google Agent Development Kit (ADK) orchestrator.³ This orchestrator manages the multi-step reasoning loop, dictating how the Gemini 2.0 Flash model processes the data.¹ The Gemini model is tasked with correlating the raw signals, classifying the incident type, determining the root cause, and assigning a severity score.¹ Operating alongside the LLM is a deterministic Policy Engine. This engine enforces hard rules, Service Level Agreement (SLA) timers, and strict governance gates, ensuring that the AI cannot circumvent human approval for high-risk actions.¹

The Memory and Self-Evaluation Layer provides the agent with historical context and persistence. This layer is heavily reliant on an Elastic knowledge base, which utilizes semantic vector search to retrieve historical runbooks and past incident metadata.¹ Structured incident metadata, state tracking, and relational data are stored within a PostgreSQL database, serving as the primary incident store.¹ Additionally, this layer manages the Slack bot interface, serving as the asynchronous approve and reject gate where human engineers interact with the agent's proposals.¹

Finally, the Observe Layer provides continuous telemetry on the agent's performance. Arize Phoenix operates here as the ultimate arbiter of system reliability.¹ It captures granular traces of every reasoning step, executes LLM-as-a-judge evaluations to grade the quality of the agent's outputs, and facilitates the adaptive autonomy loop.¹ By querying this layer, the agent can dynamically adjust its own confidence scores based on empirical historical accuracy.¹

Distributed Systems Trade-offs and CAP Theorem Integration

In distributed systems, the CAP Theorem dictates that a network must trade off between Consistency and Availability during a partition.² The OpsSentinel architecture navigates this by applying different trade-offs to different layers. The alert ingestion mechanism via Pub/Sub is tuned for Availability and Partition Tolerance (AP).² It must remain highly available to accept all incoming alerts from external monitoring tools, even if the database indexing those alerts falls momentarily behind, ensuring no critical telemetry is dropped.²

Conversely, the remediation execution and PostgreSQL incident store enforce Consistency and Partition Tolerance (CP).² When the agent attempts to update a ticket, alter the state of an incident, or execute a database script, the system enforces strong consistency to prevent race conditions or duplicate remediation executions.² To guarantee the requisite 99.99% availability for these persistent data stores, the PostgreSQL database utilizes an Active-Passive (Master-Slave) replication topology.² Heartbeats are monitored continuously; if the primary instance fails, a passive standby instance assumes the primary IP address to resume operations with minimal downtime.² Furthermore, placing redundant Cloud Run compute regions in parallel drastically increases fault tolerance, as overall availability increases mathematically when components are arranged in parallel.²

Technology Stack Decisions and Rationale

The selection of the technology stack is driven by the requirements for high concurrency, seamless AI integration, and enterprise-grade security. Table 4 outlines the core infrastructure choices and their justifications.

Architectural Layer	Selected Technology	Strategic Rationale
Core Backend	Python	Required for native compatibility with Google

		ADK, LangChain ecosystem, and advanced AI instrumentation libraries. ¹
Event Queue	Google Cloud Pub/Sub	Provides infinitely scalable asynchronous event buffering with guaranteed delivery, essential for mitigating alert storms. ¹
Relational Database	PostgreSQL	Ensures strong consistency (ACID compliance) for structured metadata, state management, and user configurations. ¹
Agent Framework	Google ADK (Graph Workflows)	Orchestrates multi-step reasoning. Graph workflows are preferred over template loops for precise, deterministic control. ¹
Compute Hosting	Google Cloud Run	Serverless execution scales dynamically from zero to thousands of instances during major outages, minimizing idle costs. ¹
Vector Engine	Elasticsearch	Provides robust hybrid search capabilities, combining dense vector embeddings with traditional full-text keyword retrieval. ⁵

Frontend Strategy and Next.js Deployment Best Practices

While the backend handles asynchronous orchestration, the frontend serves as the management and observability console for engineering directors and SREs. The frontend is built using Next.js with the App Router, optimized for serverless deployments on Google Cloud Run.¹⁵ To ensure minimal cold start times and maximal security on Cloud Run, the Next.js application must be containerized using a highly optimized, multi-stage Docker build.¹⁷ Industry standards dictate the use of Dockerfiles over Buildpacks for applications requiring specific

system-level dependencies or granular optimization.¹³

The Docker configuration utilizes Alpine Linux as the base image to dramatically reduce the container footprint, minimizing bandwidth consumption and accelerating deployment pipelines.¹⁷ Next.js is configured in standalone output mode, which instructs the compiler to trace the application dependencies and bundle only the exact files required for production execution, further shrinking the image size.¹⁸ From a security posture, the Dockerfile explicitly creates and utilizes a non-root user account, a foundational best practice that mitigates privilege escalation vulnerabilities.¹⁸

The deployment of this containerized frontend on Google Cloud requires specific networking configurations. A Serverless Network Endpoint Group (NEG) is created to securely connect the external application load balancer to the Cloud Run service.¹⁸ The load balancer utilizes a Layer 7 routing protocol to distribute incoming client requests based on application-layer information such as HTTP headers and cookies.² To optimize the delivery of static assets, the Next.js frontend integrates with Google Cloud CDN using a Pull CDN strategy. When the first user requests an asset, the CDN pulls it from the Cloud Run origin and caches it based on Time-to-Live (TTL) headers, minimizing origin traffic and providing low-latency asset delivery for all subsequent users.²

Authentication and identity management for the dashboard rely on Google Identity Services.²¹ ID tokens retrieved on the client side are transmitted to the backend, where they are cryptographically verified using official Google API client libraries to prevent spoofing.²¹ The system uses the subject claim from the decoded token as the immutable primary key for user mapping, ensuring strict separation of authentication protocols from infrastructure authorization scopes.²⁰

Data Sources, Integrations, and Agentic Memory

The operational capability of OpsSentinel is fundamentally powered by its ability to interface with external systems without writing brittle, custom API wrappers. This is achieved through the Model Context Protocol (MCP).⁴ MCP operates on a client-host-server architecture, standardizing how Large Language Models query data and execute commands across disparate platforms.⁴

The ADK framework natively supports MCP via the `McpToolset` class. The OpsSentinel agent operates as the MCP Client, connecting to external MCP Servers hosted in independent Docker containers.⁶ Using Python, the connection is established via Server-Sent Events (SSE), enabling real-time, persistent HTTP streaming from the server to the client.⁷ This protocol greatly improves error handling and deployment flexibility compared to legacy standard input/output methods.⁷

Elastic: Semantic Memory and Vector Search

Elasticsearch serves a dual purpose: acting as the high-throughput intake layer for raw logs, and serving as the long-term semantic memory repository for the agent.¹ To overcome the limitations of exact-keyword searches, historical runbooks and incident summaries are

embedded into a vector database using models optimized for speed and accuracy, specifically the all-MiniLM-L6-v2 model. This model offers a highly efficient 384-dimensional space with 100ms encoding times, representing the ideal intersection of latency and precision for operational logs.¹⁴

When the Gemini reasoning engine requires historical context, it issues a dense vector search via the Elastic MCP server.⁵ The system calculates similarities using K-Nearest Neighbors, combining these results with traditional full-text relevance scoring via Reciprocal Rank Fusion (RRF).¹⁴ This allows the agent to retrieve a past incident describing a "database connection limit reached" even if the current alert is phrased as a "connection pool exhausted".¹

Arize Phoenix: LLM Observability and Tracing

Trust is the primary bottleneck preventing autonomous AI remediation in enterprise environments.¹ OpsSentinel integrates Arize Phoenix not merely as a passive dashboard, but as an active component of the agent's cognitive loop.¹ The ADK application is instrumented using the OpenTelemetry-compatible openinference-instrumentation-google-adk package.¹⁰ This guarantees that every tool call, context retrieval, and generated token is logged as a discrete trace span within the Phoenix Cloud.⁹

The innovation lies in the feedback mechanism. Prior to executing a remediation, the agent uses the Arize MCP server to query its own historical traces.¹ If Phoenix reports that the agent's LLM-as-a-judge accuracy score for the current category has degraded, the agent programmatically lowers its autonomy tier, refusing to act without explicit human intervention.¹ This continuous evaluation ensures that the agent's performance remains highly visible and trustworthy over time.¹

Dynamic ADK Graph Workflows

Earlier iterations of ADK relied on Template Workflows, such as sequential or loop agents, which rigidly executed tasks in predefined chains.²⁷ While useful for simple tasks, OpsSentinel requires deterministic conditional logic that scales with incident complexity. Therefore, the architecture leverages ADK 2.0 Graph-based Workflows.³⁰ Graph workflows define explicit edges and nodes, permitting the orchestration engine to bypass the LLM for strictly deterministic tasks, such as parsing an incoming JSON webhook payload.³⁰ The Gemini model is invoked exclusively for complex reasoning, such as synthesizing the root cause from disparate logs.³⁰ This selective routing minimizes token consumption, drastically reduces latency, and severely mitigates the risk of hallucination during trivial data transformation stages.¹²

Systemic Risks, Unknowns, and Security Protocols

Deploying agentic AI with write-access to production infrastructure introduces significant operational risks. These are mitigated through strict engineering constraints and governance policies. Table 5 outlines the primary risks and their corresponding systemic mitigations.

Operational Risk Factor	Consequence	Engineered Mitigation Strategy
LLM Hallucinations	Agent invents phantom commands or references non-existent infrastructure components.	Enforcement of strict Retrieval-Augmented Generation (RAG) via Elastic. The agent cannot act outside the bounds of retrieved historical runbooks. ³⁰
Alert Correlation Failure	The system fails to recognize related alerts, continuing the pattern of duplicate ticketing.	Implementation of time-windowed spatial correlation algorithms before LLM invocation, ensuring related signals are pre-grouped based on service IDs. ¹
Inference Cost Explosions	Runaway LLM execution loops consume massive amounts of API billing during a storm.	Utilization of deterministic ADK graph workflows to prevent infinite agent loops, coupled with hard Google Cloud Scheduler API quotas. ¹
Context Retrieval Latency	Semantic searches take too long, violating the strict 30-second MTTD metric.	Optimization of Elasticsearch indexing, utilizing the lightweight all-MiniLM-L6-v2 embedding model over larger, slower counterparts. ¹⁴

Furthermore, the principle of least privilege is strictly enforced across all service accounts executing the Docker containers and ADK agents.²⁰ Broad IAM roles, such as the default project Editor, are explicitly prohibited. Instead, custom Identity and Access Management (IAM) roles restrict the agent to specific, non-destructive actions unless explicitly authorized by a senior engineer via the Slack webhook.³² API keys for Gemini, Elasticsearch, and Arize are never hardcoded or committed to version control; they are injected dynamically at runtime via Google Cloud Secret Manager, ensuring cryptographic security at rest and in transit.⁸

Implementation Milestones and Build Phases

Execution of the MVP must occur in strictly managed, iterative phases to contain technical debt and validate core integrations early in the development lifecycle. This structured approach aligns with lean engineering principles. Table 6 delineates the implementation phases.

Phase	Core Objective	Technical Deliverables
Phase 1	Infrastructure and Orchestration Foundation	Deploy Google Cloud Run instances, configure the Pub/Sub event bus, and provision Secret Manager for credential storage. ¹
Phase 2	Data Sources and MCP Server Deployment	Containerize and deploy Elastic and Arize MCP servers. Configure vector indices and test semantic search using synthetic telemetry. ⁸
Phase 3	ADK Agent Architecture	Develop the Python ADK application. Implement Graph Workflow orchestrator to define execution nodes and integrate MCP Toolsets. ⁴
Phase 4	Frontend Development and Security	Develop the Next.js dashboard. Containerize using a multi-stage Alpine Dockerfile. Implement Google Identity Services for authentication. ¹⁸
Phase 5	Workflow Integration and Feedback Loop	Connect the Slack integration for approval webhooks. Run end-to-end simulations of cascading alert storms to validate deduplication logic. ¹

Exit Criteria and Validation Checkpoints

The MVP development cycle is considered complete, and the project is authorized for internal beta deployment, only when rigorous empirical criteria are met. This prevents the endless engineering cycles common in complex AI deployments.

The system must successfully demonstrate end-to-end tracing during a simulated stress test. A synthetic alert storm consisting of no fewer than fifty duplicate signals must be ingested via the Pub/Sub queue, dynamically deduplicated, and synthesized into a single incident entity without a single dropped event, proving the system's high availability and scalability under load.¹

The autonomous evaluation mechanisms must function flawlessly. The Gemini ADK agent must successfully retrieve a mathematically relevant historical runbook via the Elastic MCP server, query its own baseline performance metrics via the Arize Phoenix MCP server, and accurately format a remediation proposal based on this synthesized data.¹ This proposal must be routed to a designated Slack channel, where a human operator clicking the approval button triggers a deterministic execution path that correctly updates the mocked infrastructure state and resolves the ticket.¹

Finally, observability must be absolute. Every phase of the stress test must be accurately represented as an OpenInference trace within the Arize Phoenix user interface, clearly delineating tool executions, latency metrics, and token consumption.⁹ Concurrently, the Next.js frontend must securely load behind Google Authentication protocols, rendering the incident state via server-side data fetching with zero unhandled client-side exceptions.²¹ Meeting these criteria signifies a viable, robust, and highly reliable initial product offering ready for operational testing.